



Week 1 Progress Report

Road Pothole Detection System Using YOLO-Based Deep Learning

Programme: B.Tech CSE (AIML)

Group: AIML74

Mentor : Mritunjay Kumar

UPES, Dehradun

Team Members

Roll No.	Name	Responsibility
500119453	Aditya Khatri	Model training, evaluation and comparison of YOLOv11n vs YOLOv12n
500119770	Ujjawal Gupta	Dataset collection, preprocessing and annotation conversion to YOLO format
500126591	Arsh Rathi	Literature review, documentation and report preparation

1. Introduction

One of the ongoing issues that both local governments and regular commuters deal with is the deterioration of road infrastructure. In instance, potholes are a common issue that goes beyond simple annoyance; they can result in tire blowouts, vehicle suspension damage, collisions, and even fatalities. Despite their influence, most cities still primarily rely on human reporting or reactive maintenance for pothole discovery, neither of which is scalable or timely.

The goal of this project is to use deep learning, specifically the YOLO (You Only Look Once) family of object detection models, to create an automated pothole detection system. The concept is simple: a camera put at a roadside position or mounted on a vehicle may continuously scan the road surface and identify potholes in real time, eliminating the need for human inspectors or citizen complaints. After that, the output can either be recorded for future review or sent into a road maintenance management system.

The foundation for the project was established during the first week: the problem scope was established, pertinent literature was examined, the repository was organized, and a preliminary YOLOv11 training run was finished. Everything completed thus far and what is scheduled for the upcoming weeks are included in this report.

2. Problem Statement

The automatic detection of potholes from road photos and video recordings is the problem being tackled here. A raw picture or a video frame taken from a stationary camera or a moving car serves as the system's input; the output is the same image with bounding boxes around identified potholes and a confidence score.

The difference in how potholes appear under various circumstances is what makes this non-trivial. A pothole on a sunny, dry day appears quite different than one on a road that is wet or cloudy. Similarly, how well the pothole appears in the picture depends on a number of factors, including the camera angle, vehicle speed, illumination, and the roughness of the surrounding road. Careful dataset selection and augmentation techniques are necessary for a strong detection model to generalize across these settings.

Speed is the second challenge. A detecting system must analyze frames quickly enough to be regarded as real-time—ideally 25 frames per second or more—in order to be realistically relevant in a moving vehicle. Lightweight YOLO architectures like YOLOv11n and YOLOv12n are being investigated instead of heavier models because this introduces a trade-off between model accuracy and computing efficiency.

3. Project Objectives

The broad aim of this project is to build, evaluate, and compare pothole detection models based on YOLOv11 and YOLOv12 architectures. More specifically, the work is structured around the following goals:

1. Train YOLOv11n and YOLOv12n on a publicly available, labelled pothole dataset and compare their detection performance using standard metrics (mAP, precision, recall).
2. Build an inference pipeline capable of processing both static images and video files, producing annotated outputs saved to disk.
3. Investigate how well the trained models generalise to unseen road conditions, particularly scenarios not well-represented in the training data.
4. Document the training process, configuration decisions, and evaluation results in a way that is reproducible by others.
5. Lay the groundwork for potential deployment, for instance as a dashcam-based detection tool or integration into a road maintenance workflow.

4. Functional Requirements

The system being developed has a number of concrete capabilities it needs to support. These are described below, grouped by area.

4.1 Detection and Inference

Fundamentally, the system must be able to generate an annotated output with pothole locations indicated by bounding boxes from an input, which could be a single picture file or a video. The model's confidence score and class label should be reported by the system after every detection. This is handled by the inference script (`run_detector.py`), which is made to function with both `mp4` video files and `png/.jpg` images.

The system scans the video input frame by frame and overlays the detections on an output video (`pothole_detection_result.mp4`). In order for the output video to play back smoothly, the target inference speed for a mid-range GPU is at least 25 frames per second. This will inevitably be slower when using just the CPU, but the output quality should not change.

4.2 Model Training

A specific script called `train_yolov12.py` and a dataset configuration file called `pothole_dataset.yaml` are used to oversee the training portion of the project. The training pipeline must be able to import pre-trained weights (`yolo11n.pt` and `yolov12n.pt` are included in the repository) and use transfer learning to fine-tune them on the pothole dataset. This is crucial since poor generalization is frequently the result of training from start on a tiny sample.

Weights and metric logs are among the training outputs that are automatically kept in the `runs/detect/` directory. Key performance metrics are summarized and written to `REPORT.txt` at the conclusion of the session. Without having to modify the main model code, the pipeline should make it simple to change hyperparameters like the number of epochs, batch size, and input image resolution.

4.3 Dataset Handling and Preprocessing

Depending on the source, the raw dataset may arrive in several annotation forms. Annotations are converted into the YOLO text format required by the training pipeline by a conversion script (`convert_pothole_to_yolo.py`). The class ID and the normalized bounding box coordinates are specified on a single line per item in each annotation file.

Images are scaled to 640×640 pixels, the typical input size for YOLO models, before training. During training, common augmentations including horizontal flipping, HSV color jitter, mosaic patching, and random scaling are used to increase the model's resilience. The YAML file configures the routes to the training, validation, and test subsets of the dataset.

4.4 Output and Reporting

Image detection results are stored in the `outputs/` directory. The system creates a basic record of what was found along with annotated versions of the input files for every run. Following training, the final mAP scores, precision, recall, and, if relevant, a comparison between the two YOLO versions are recorded in the `REPORT.txt` file. Because of this, summarizing results is simple and doesn't require running the models again.

5. Dataset Requirements

5.1 Why Dataset Choice Matters Here

The quality and variety of the data used to train object detection algorithms have a significant impact on their performance. This is especially important for pothole detection because the size, shape, depth, and appearance of potholes vary greatly depending on the kind of road surface, the weather, and the angle at which they are taken. When tested on photographs from several locations, a model that was only trained on images from one kind of road or one geographic area frequently performs poorly.

The objective of this project is to combine publicly accessible labeled datasets with locally recorded road footage in and around Meerut, Uttar Pradesh, whenever feasible. This gives ground-truth data that is directly pertinent to the deployment context. The conversion script will be used to convert the local data to YOLO format after it has been manually annotated using a program such as LabelImg.

5.2 Real-World Datasets Under Consideration

Dataset	Source	Approx. Size	Remarks
---------	--------	--------------	---------

Pothole Detection Dataset	Roboflow Universe	~780 images	Pre-labelled in YOLO format; varied lighting and road types
Road Damage Dataset 2022 (RDD2022)	IEEE DataPort	~47,000 images	Multi-country; includes potholes, cracks, rutting across Japan, India, USA, Norway
Pothole-600	GitHub / Academic	~600 images	Focused specifically on potholes; commonly used as a benchmark
UA-DETRAC	DETRAC Project	~140,000 frames	Traffic video dataset; useful for evaluating inference on video streams
Custom Local Footage	Dashcam / Mobile	To be collected	Roads around Meerut and nearby areas; annotated manually

5.3 Annotation Format and Split

Prior to training, all datasets are transformed into the YOLO annotation format. Every image has a matching.txt file with one line for each annotated object: the class index (0 for potholes), followed by the bounding box's dimensions and normalized center coordinates. The dataset is divided into roughly 20% validation, 10% test, and 70% training images. The training script automatically reads these paths, which are specified in pothole_dataset.yaml.

5.4 Minimum Data Quality Standards

There must be at least 500 identified pothole events throughout the training split for the training data to be useful. In order to prevent the model from overfitting to a limited range of appearances, images should represent a reasonable variety of situations, such as different times of day, wet and dry surfaces, urban and highway road types. Every image should be at least 640 by 480 pixels in size. Blurry, severely occluded, or duplicate training image frames are not included.

6. Tools and Technology

The project is written entirely in Python and makes use of the Ultralytics library, which provides a clean, well-maintained implementation of the YOLO model family. PyTorch serves as the underlying deep learning framework. OpenCV is used for image and video I/O as well as for drawing bounding boxes on output frames.

Area	Tool / Library
Language	Python 3.10+
Deep Learning	PyTorch 2.x
YOLO Models	Ultralytics — YOLOv11n and YOLOv12n
Computer Vision	OpenCV (cv2)

Dataset Management	Roboflow, custom YAML configuration
Training Environment	Local GPU machine, Google Colab, Kaggle Notebooks
Version Control	Git and GitHub

7. Work Completed in Week 1

The first week was primarily about setting the foundation. A significant portion of the time went into reading existing work on YOLO-based detection for road anomalies, which helped clarify which model variants were worth evaluating and what dataset sizes are typically needed to achieve reasonable accuracy.

An initial training run was completed on a small subset of the Roboflow dataset using YOLOv11n to verify that the pipeline was working end-to-end. The inference script was then used to run detection on a sample image and a short video clip, both of which produced plausible detections, although accuracy at this stage is low due to the limited training data.

8. Plan for Week 2

The priority in the coming week is to move from the proof-of-concept stage to a full training run. The complete Roboflow pothole dataset will be used along with a portion of the RDD2022 dataset, bringing the total labelled instance count above the 500-image threshold. Augmentation settings will be reviewed and adjusted to improve the diversity of the training samples.

Both YOLOv11n and YOLOv12n will be trained for at least 100 epochs, and their validation metrics will be logged and compared. If time permits, some local footage will be collected and added to the dataset to begin testing how well the models transfer to Indian road conditions. The results will be written up in REPORT.txt and reviewed with the supervisor.

9. Challenges Encountered and Anticipated

The scarcity of labeled training data is the most pressing practical issue. About 780 photos make up the Roboflow dataset, which is manageable yet little by today's standards. When the model is trained solely on this, it runs the risk of becoming overconfident in the visual patterns it has seen and having trouble with images that differ even slightly. Aggressive augmentation and the incorporation of data from other sources will be used to partially remedy this.

Another issue is computational resources. Even with a GPU, training YOLO for 100 or more epochs takes a significant amount of time. Google Colab or Kaggle Notebooks will be utilized to augment local compute for longer runs. In the event that a session times out, care will be made to save checkpoints at regular intervals so that training can restart without having to start again.

Differentiating potholes from other road surface problems, such as cracks, rutting, and patches, which might appear similar in some lighting conditions, is a more delicate problem. Because only potholes

are labeled in the current dataset, the model may generate false positives for cracks that are accidentally captured from a perspective that makes them appear pit-like. During validation, this is something to keep a close eye on.

10. References

1. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of CVPR 2016.
2. Ultralytics (2024). YOLO Documentation. Retrieved from <https://docs.ultralytics.com>
3. Arya, D., et al. (2022). RDD2022: A multi-national image dataset for automatic Road Damage Detection. IEEE DataPort.
4. Roboflow Pothole Detection Dataset. Retrieved from <https://universe.roboflow.com>
5. OpenCV Documentation. Retrieved from <https://docs.opencv.org>
6. Jocher, G., et al. (2023). Ultralytics YOLOv8. GitHub. <https://github.com/ultralytics/ultralytics>